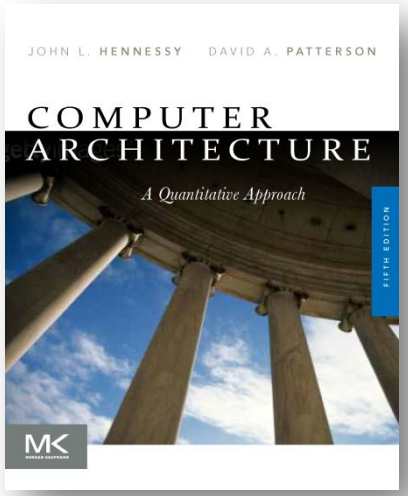


Unit - 1

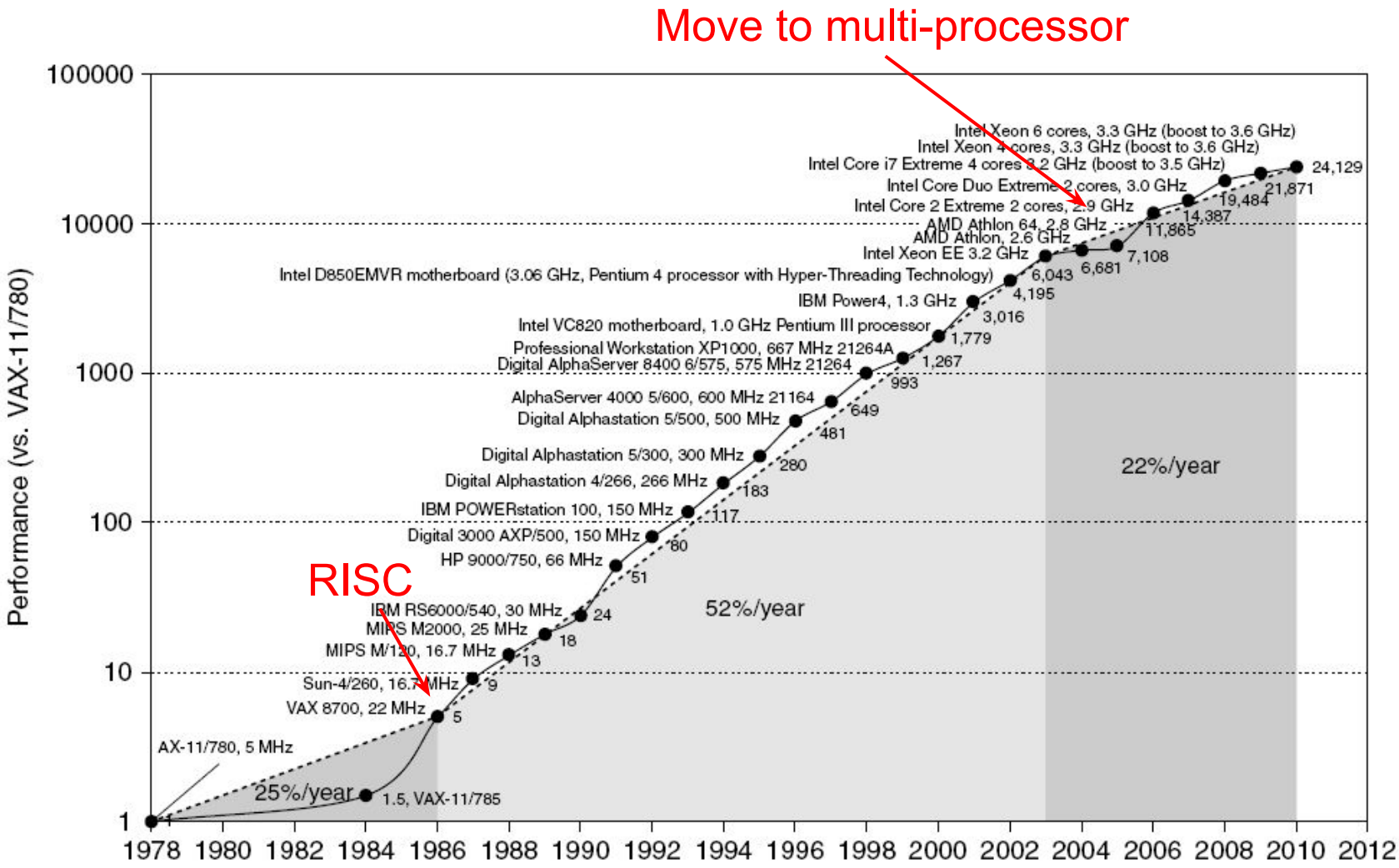
Fundamentals of Quantitative Design and Analysis

By
Dr. Sandhya . S
Dept. of CSE, RVCE

Computer Technology

- Performance improvements:
  - Improvements in semiconductor technology
    - Feature size, clock speed
  - Improvements in computer architectures
    - Enabled by HLL compilers, UNIX
    - Lead to RISC architectures
  - Together have enabled:
    - Lightweight computers
    - Productivity-based managed/interpreted programming languages

Single Processor Performance



Current Trends in Architecture

- Cannot continue to leverage Instruction-Level parallelism (ILP)
  - Single processor performance improvement ended in 2003
- New models for performance:
  - Data-level parallelism (DLP)
  - Thread-level parallelism (TLP)
  - Request-level parallelism (RLP)
- These require explicit restructuring of the application

Classes of Computers

- Personal Mobile Device (PMD)
  - e.g. start phones, tablet computers
  - Emphasis on energy efficiency and real-time
- Desktop Computing
  - Emphasis on price-performance
- Servers
  - Emphasis on availability, scalability, throughput
- Clusters / Warehouse Scale Computers
  - Used for “Software as a Service (SaaS)”
  - Emphasis on availability and price-performance
  - Sub-class: Supercomputers, emphasis: floating-point performance and fast internal networks
- Embedded Computers
  - Emphasis: price

Parallelism

- Classes of parallelism in applications:
  - Data-Level Parallelism (DLP)
  - Task-Level Parallelism (TLP)

- Classes of architectural parallelism:
  - Instruction-Level Parallelism (ILP)
  - Vector architectures/Graphic Processor Units (GPUs)
  - Thread-Level Parallelism
  - Request-Level Parallelism

Flynn's Taxonomy

- Single instruction stream, single data stream (SISD)
- Single instruction stream, multiple data streams (SIMD)
  - Vector architectures
  - Multimedia extensions
  - Graphics processor units
- Multiple instruction streams, single data stream (MISD)
  - No commercial implementation
- Multiple instruction streams, multiple data streams (MIMD)
  - Tightly-coupled MIMD
  - Loosely-coupled MIMD

Defining Computer Architecture

- "Old" view of computer architecture:
  - Instruction Set Architecture (ISA) design
  - i.e. decisions regarding:
    - registers, memory addressing, addressing modes, instruction operands, available operations, control flow instructions, instruction encoding
- "Real" computer architecture:
  - Specific requirements of the target machine
  - Design to maximize performance within constraints: cost, power, and availability
  - Includes ISA, microarchitecture, hardware

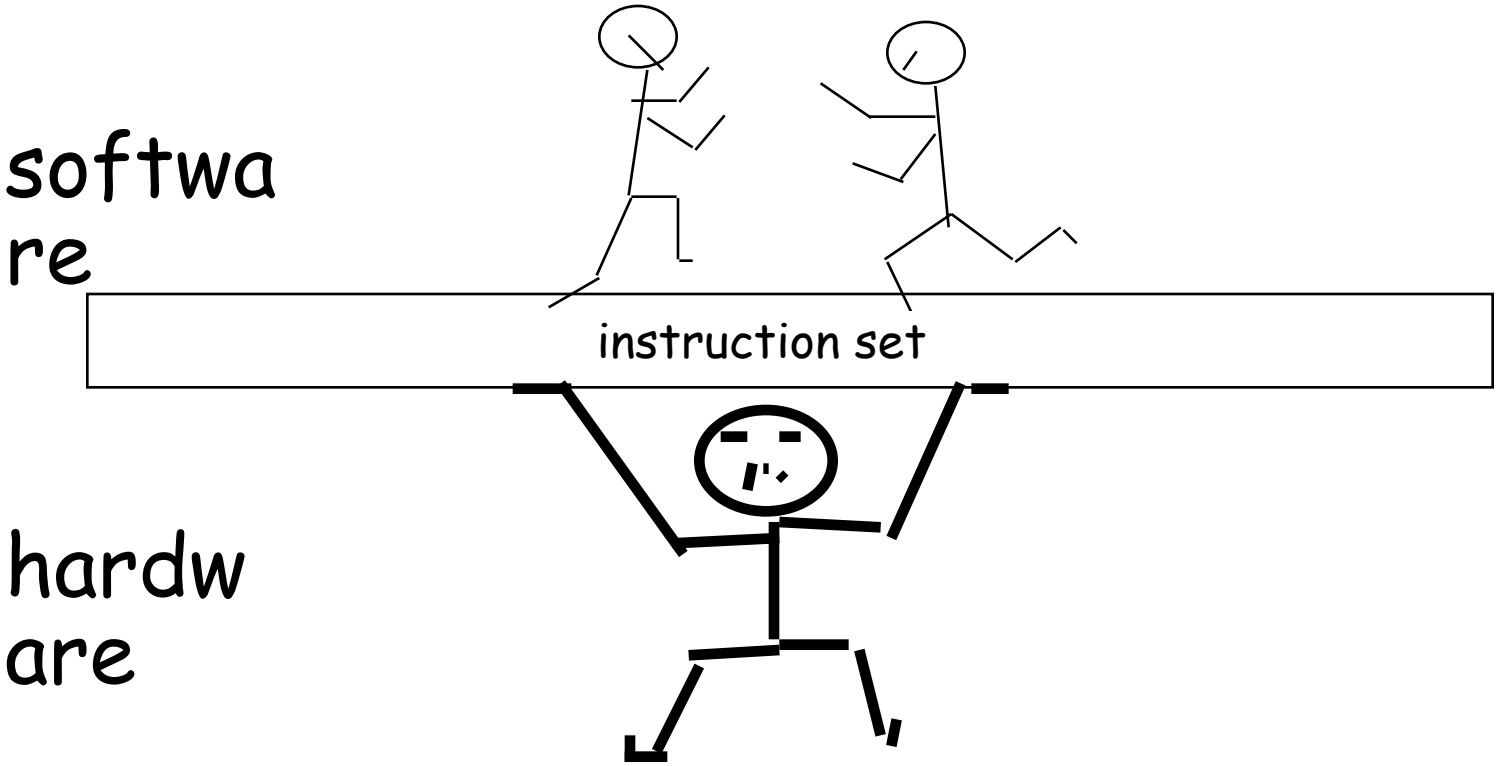
What is *Computer Architecture*

Computer Architecture =

Instruction Set Architecture +

Organization + Hardware + ...

The Instruction Set: a Critical Interface

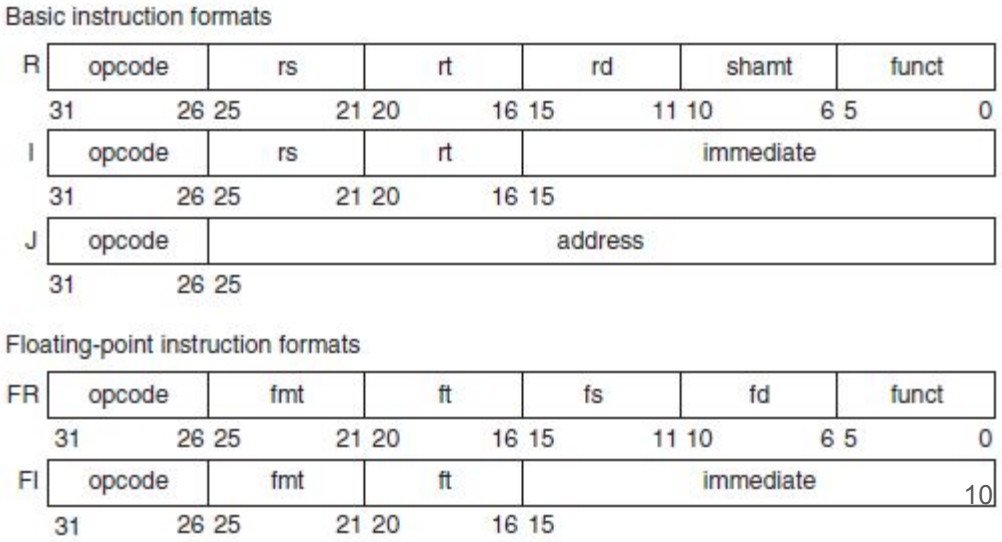


Defining Computer Architecture

The seven dimensions of an ISA:

1. ***Class of ISA –*** register memory ISA's & load-store ISA's
2. ***Memory addressing –*** byte addressing / aligned
3. ***Addressing modes –*** register, immediate & displacement
4. ***Types and sizes of operands –*** 8/16/32/64-bit etc.,
5. ***Operations-***data transfer, arithmetic logical, control & FP.
6. ***Control flow instructions -*** conditional branches, unconditional jumps, procedure calls, and returns
7. ***Encoding an ISA –***

fixed length and variable length.



Defining Computer Architecture

Instruction type/opcode	Instruction meaning
Data transfers	*Move data between registers and memory, or between the integer and FP or special registers; only memory address mode is 16-bit displacement + contents of a GPR*
LB, LBU, SB	Load byte, load byte unsigned, store byte (to/from integer registers)
LH, LHU, SH	Load half word, load half word unsigned, store half word (to/from integer registers)
LW, LWU, SW	Load word, load word unsigned, store word (to/from integer registers)
LD, SD	Load double word, store double word (to/from integer registers)
L.S, L.D, S.S, S.D	Load SP float, load DP float, store SP float, store DP float
MFC0, MTC0	Copy from/to GPR to/from a special register
MOV.S, MOV.D	Copy one SP or DP FP register to another FP register
MFC1, MTC1	Copy 32 bits to/from FP registers from/to integer registers
Arithmetic/logical	*Operations on integer or logical data in GPRs: signed arithmetic trap on overflow*
DADD, DADDI, DADDU, DADDIU	Add, add immediate (all immediates are 16 bits); signed and unsigned
DSUB, DSUBU	Subtract, signed and unsigned
DMUL, DMULU, DDIV, DDIVU, MADD	Multiply and divide, signed and unsigned; multiply-add; all operations take and yield 64-bit values
AND, ANDI	And, and immediate
OR, ORI, XOR, XORI	Or, or immediate, exclusive or, exclusive or immediate
LUI	Load upper immediate; loads bits 32 to 47 of register with immediate, then sign-extends
DSLL, DSRL, DSRA, DSLLV, DSRLV, DSRAV	Shifts: both immediate (DS__) and variable form (DS__V); shifts are shift left logical, right logical, right arithmetic
SLT, SLTI, SLTU, SLTIU	Set less than, set less than immediate, signed and unsigned
Control	*Conditional branches and jumps; PC-relative or through register*
BEQZ, BNEZ	Branch GPRs equal/not equal to zero; 16-bit offset from PC + 4
BEQ, BNE	Branch GPR equal/not equal; 16-bit offset from PC + 4
BC1T, BC1F	Test comparison bit in the FP status register and branch; 16-bit offset from PC + 4
MOVN, MOVZ	Copy GPR to another GPR if third GPR is negative, zero
J, JR	Jumps: 26-bit offset from PC + 4 (J) or target in register (JR)
JAL, JALR	Jump and link: save PC + 4 in R31, target is PC-relative (JAL) or a register (JALR)
TRAP	Transfer to operating system at a vectored address
ERET	Return to user code from an exception; restore user mode
Floating point	*FP operations on DP and SP formats*
ADD.D, ADD.S, ADD.PS	Add DP, SP numbers, and pairs of SP numbers
SUB.D, SUB.S, SUB.PS	Subtract DP, SP numbers, and pairs of SP numbers
MUL.D, MUL.S, MUL.PS	Multiply DP, SP floating point, and pairs of SP numbers
MADD.D, MADD.S, MADD.PS	Multiply-add DP, SP numbers, and pairs of SP numbers
DIV.D, DIV.S, DIV.PS	Divide DP, SP floating point, and pairs of SP numbers
CVT._._	Convert instructions: CVT.x.y converts from type x to type y, where x and y are L (64-bit integer), W (32-bit integer), D (DP), or S (SP). Both operands are FPRs.
C.__.D, C.__.S	DP and SP compares: "__" = LT,GT,LE,GE,EQ,NE; sets bit in FP status register

Dependablility

- ***Service Level Agreements* (SLAs) / *Service Level Objectives* (SLOs)**
- ***Systems alternate between two states of service with respect to an SLA:***
    1. ***Service accomplishment***
    2. ***Service interruption***

Transitions between these two states are caused by failures / restorations.

- ***Quantifying these transitions leads to the two main measures of dependability:***
    1. ***Module reliability***
    2. ***Module availabilitv***

$$\text{Module availability} = \frac{\text{MTTF}}{(\text{MTTF} + \text{MTTR})}$$

Reliability and availability are now quantifiable metrics

Problem4:

Assume a disk subsystem with the following components and MTTF:

- 10 disks, each rated at 1,000,000-hour MTTF
- 1 ATA controller, 500,000-hour MTTF
- 1 power supply, 200,000-hour MTTF
- 1 fan, 200,000-hour MTTF
- 1 ATA cable, 1,000,000-hour MTTF

Using the simplifying assumptions that the lifetimes are exponentially distributed and that failures are independent, compute the MTTF of the system as a whole.

Ans: The sum of the failure rates is

$$\text{Failure rate}_{\text{system}} = 10 \times \frac{1}{1{,}000{,}000} + \frac{1}{500{,}000} + \frac{1}{200{,}000} + \frac{1}{200{,}000} + \frac{1}{1{,}000{,}000}$$

$$= \frac{10 + 2 + 5 + 5 + 1}{1.000.000 \text{ hours}} = \frac{23}{1.000.000} = \frac{23{,}000}{1.000.000.000 \text{ hours}}$$

23,000 FIT. The MTTF for the system is just the inverse of the failure rate:

$$\text{MTTF}_{\text{system}} = \frac{1}{\text{Failure rate}_{\text{system}}} = \frac{1{,}000{,}000{,}000 \text{ hours}}{23{,}000} = 43{,}500 \text{ hours}$$

Measuring Performance

- Typical performance metrics:
  - Response time
  - Throughput

- Speedup of X relative to Y
  - Execution time$_Y$ / Execution time$_X$ i.e.,

- Execution time
  - Wall clock time: includes all system ov
  - CPU time: only computation time

- Benchmarks
  - Kernels (e.g. matrix multiply)
  - Toy programs (e.g. sorting)
  - Synthetic benchmarks (e.g. Dhrystone)
  - Benchmark suites (e.g. SPEC06fp, TPC-C)
    www.spec.org

Since execution time is the reciprocal of performance:

$$n = \frac{\text{Execution time}_Y}{\text{Execution time}_X} = \frac{\frac{1}{\text{Performance}_Y}}{\frac{1}{\text{Performance}_X}} = \frac{\text{Performance}_X}{\text{Performance}_Y}$$

Choosing Programs to Evaluate Perf.

- Toy benchmarks
  - e.g., quicksort, puzzle
  - No one really runs. Scary fact: used to prove the value of RISC in early 80's
- Synthetic benchmarks
  - Attempt to match average frequencies of operations and operands in real workloads.
  - e.g., Whetstone, Dhrystone
  - Often slightly more complex than kernels; But do not represent real programs
- Kernels
  - Most frequently executed pieces of real programs
  - e.g., livermore loops
  - Good for focusing on individual features not big picture
  - Tend to over-emphasize target feature
- Real programs
  - e.g., gcc, spice, SPEC2006 (standard performance evaluation corporation), TPCC, TPCD, PARSEC, SPLASH

Principles of Computer Design

- Take Advantage of Parallelism
  - e.g. multiple processors, disks, memory banks, pipelining, multiple functional units
- Principle of Locality
  - Reuse of data and instructions
    - Temporal and Spatial Locality
- Focus on the Common Case

Compute Speedup – Amdahl's Law

Speedup is due to enhancement.

Amdahl's law states that,- the performance improvement to be gained from using some faster mode of execution is limited by the fraction of the time the faster mode can be used.

$$\text{Speedup} = \frac{\text{Performance for entire task using the enhancement when possible}}{\text{Performance for entire task without using the enhancement}}$$

$$\text{Speedup} = \frac{\text{Execution time for entire task without using the enhancement}}{\text{Execution time for entire task using the enhancement when possible}}$$

Amdahl's law depends on two factors:

1. The fraction of the computation time in the original computer that can be converted to take advantage of the enhancement ($\text{Fraction}_{\text{enhanced}}$, is always less than or equal to 1).

2. The improvement gained by the enhanced execution mode ($\text{Speedup}_{\text{enhanced}}$ which is always greater than 1).

Compute Speedup – Amdahl's Law

Speedup is due to enhancement(E):

$Time_{old}$

$Time_{new}$

Let F be the fraction where enhancement is applied => Also, called parallel fraction and (1-F) as the serial fraction

$$\text{Execution time}_{new} = \text{Execution time}_{old} \times \left((1 - \text{Fraction}_{enhanced}) + \frac{\text{Fraction}_{enhanced}}{\text{Speedup}_{enhanced}} \right)$$

The overall speedup is the ratio of the execution times:

$$\text{Speedup}_{overall} = \frac{\text{Execution time}_{old}}{\text{Execution time}_{new}} = \frac{1}{(1 - \text{Fraction}_{enhanced}) + \frac{\text{Fraction}_{enhanced}}{\text{Speedup}_{enhanced}}}$$

Problem5:

Suppose that we want to enhance the processor used for Web serving. The new processor is 10 times faster on computation in the Web serving application than the original processor. Assuming that the original processor is busy with computation 40% of the time and is waiting for I/O 60% of the time, what is the overall speedup gained by incorporating the enhancement?

Ans:

$\text{Fraction}_{\text{enhanced}} = 0.4;$

$\text{Speedup}_{\text{enhanced}} = 10;$

$\text{Speedup}_{\text{overall}} = ?$

$$\text{Speedup}_{\text{overall}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

$$\text{Speedup}_{\text{overall}} = \frac{1}{0.6 + \frac{0.4}{10}}$$

$$= \frac{1}{0.64}$$

$$\approx 1.56$$

Problem6:

- Suppose FP square root (FPSQR) is responsible for 20% of the execution time of a critical graphics benchmark. FPSQR hardware is enhanced to speed up this operation by a factor of 10. Also as another alternative is just to try to make all FP instructions in the graphics processor run faster by a factor of 1.6; FP instructions are responsible for half of the execution time for the application. Compare these two design alternatives.
- Ans: We can compare these two alternatives by comparing the speedups:

$$\text{Speedup}_{\text{overall}} = \frac{1}{(1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

$$\text{Speedup}_{\text{FPSQR}} = \frac{1}{(1 - 0.2) + \frac{0.2}{10}} = \frac{1}{0.82} = 1.22$$

$$\text{Speedup}_{\text{FP}} = \frac{1}{(1 - 0.5) + \frac{0.5}{1.6}} = \frac{1}{0.8125} = 1.23$$

Principles of Computer Design

- The Processor Performance Equation

$$\text{CPU time} = \text{CPU clock cycles for a program} \times \text{Clock cycle time}$$

$$\text{CPU time} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate}}$$

$$\text{CPI} = \frac{\text{CPU clock cycles for a program}}{\text{Instruction count}}$$

If we know cc and IC, we can calculate avg **CPI:**

$$\text{CPI} = \frac{\text{CPU clock cycles for a program}}{\text{Instruction count}}$$

Therefore we can write cc = IC x CPI

$$\text{CPU time} = \text{Instruction count} \times \text{Cycles per instruction} \times \text{Clock cycle time}$$

$$\frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}} = \frac{\text{Seconds}}{\text{Program}} = \text{CPU time}$$

Principles of Computer Design

- Different instruction types having different CPIs

$$\text{CPU clock cycles} = \sum_{i=1}^{n} \text{IC}_i \times \text{CPI}_i$$

$$\text{CPU time} = \left(\sum_{i=1}^{n} \text{IC}_i \times \text{CPI}_i \right) \times \text{Clock cycle time}$$

Example

- Instruction mix of a RISC architecture.

Inst.	ALU	Load	Store	Branch
Freq.	50%	20%	10%	20%
C. C.	1	2	2	2

- Add a register-memory ALU instruction format?
- One op. in register, one op. in memory
- The new instruction will take 2 cc but will also increase the Branches to 3 cc.

Q: What fraction of loads must be eliminated for this to pay off?

Solution

Instr.	F_i	CPI_i	$CPI_i \times F_i$	I_i	CPI_i	$CPI_i \times I_i$
ALU	.5	1	.5	.5-X	1	.5-X
Load	.2	2	.4	.2-X	2	.4-2X
Store	.1	2	.2	.1	2	.2
Branch	.2	2	.4	.2	3	.6
Reg/Mem				X	2	2X
	1.0		CPI=1.5	1-X		(1.7-X)/(1-X)

Exec Time = Instr. Cnt. x CPI x Cycle time

$$\text{Instr. Cnt}_{old} \times \text{CPI}_{old} \times \text{Cycle time}_{old} >= \text{Instr. Cnt}_{new} \times \text{CPI}_{new} \times \text{Cycle time}_{new}$$

$$1.0 \times 1.5 >= (1-X) \times (1.7-X)/(1-X)$$

$$X >= 0.2$$

ALL loads must be eliminated for this to be a win!

REVIEW OF PIPELINING APPENDIX C(5TH EDITION)

Pipelining

- If all stages are balanced

  i.e., all take the same time

- Time per instructions$_{pipelined}$

  = Time per instructions$_{nonpipelined}$

  Number of pipe stages

- If not balanced, speedup is less

The Basics of a RISC Instruction Set:

RISC architectures are characterized by a few key properties,-

- All operations on data apply to data in registers.
- Only load and store operations affect the memory.
- The instruction formats are few in number.

Most RISC like MIPS have 3 classes of instructions:

1. ALU Instructions
   - DADD, DSUB,(DADDU,DSUBU, DADDIU) and Logical(AND, OR
2. Load Store Instructions
   - LD or SD
3. Branches and Jumps
   - Condition codes
   - Comparison between registers / register and zero

Simple Implementation of RISC Instruction Set:

- Five clock cycles,

1. IF: Instruction Fetch Cycle
2. ID: Instruction Decode / Register Fetch Cycle

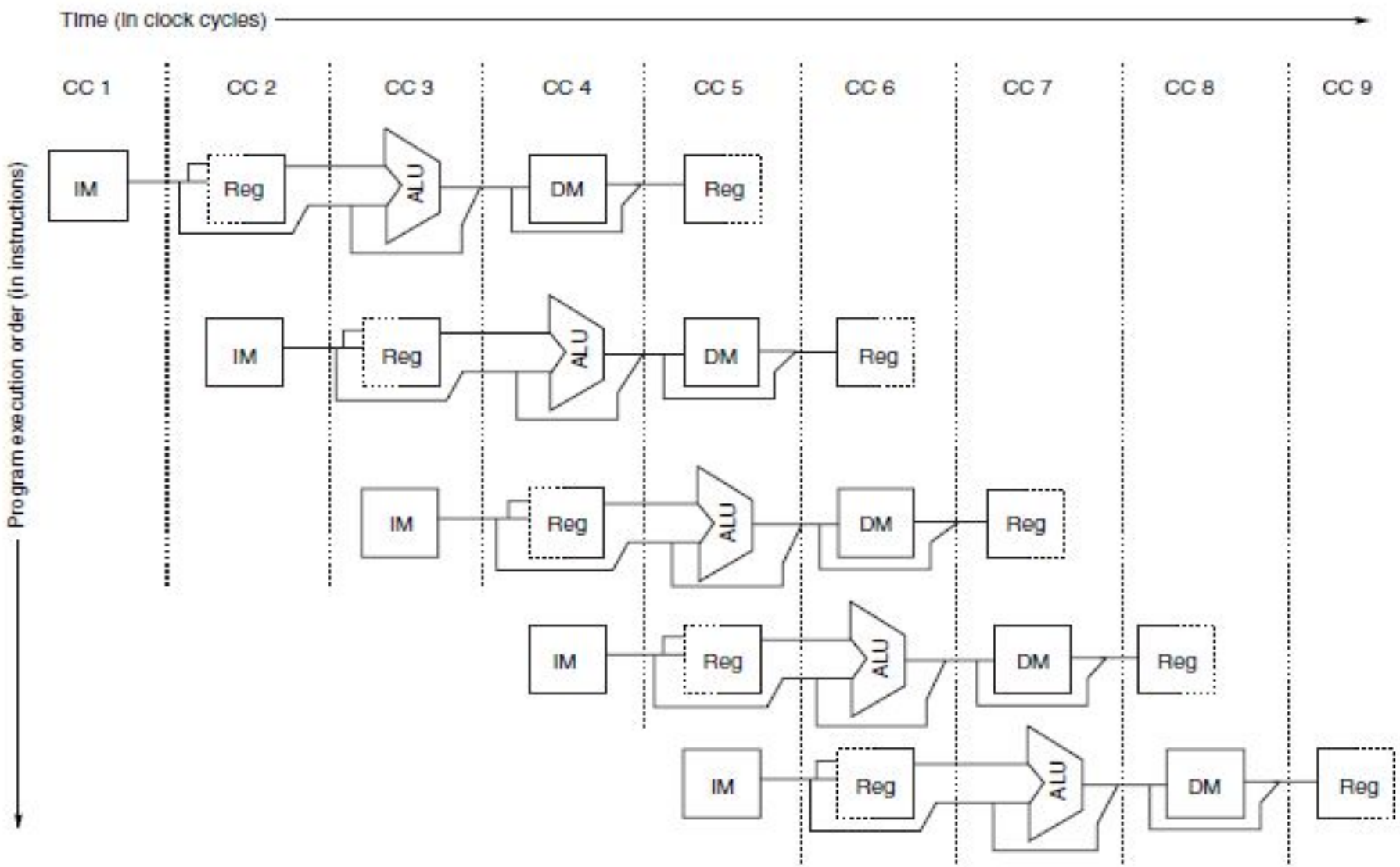
   Fixed field decoding

3. EX: Execution / Effective Address Cycle
   i. Memory Reference
   ii. Register-Register ALU Instruction
   iii. Register-Immediate ALU instruction
4. MEM: Memory Access
5. WB: Write Back cycle
   - Register-Register ALU instruction or Load instruction

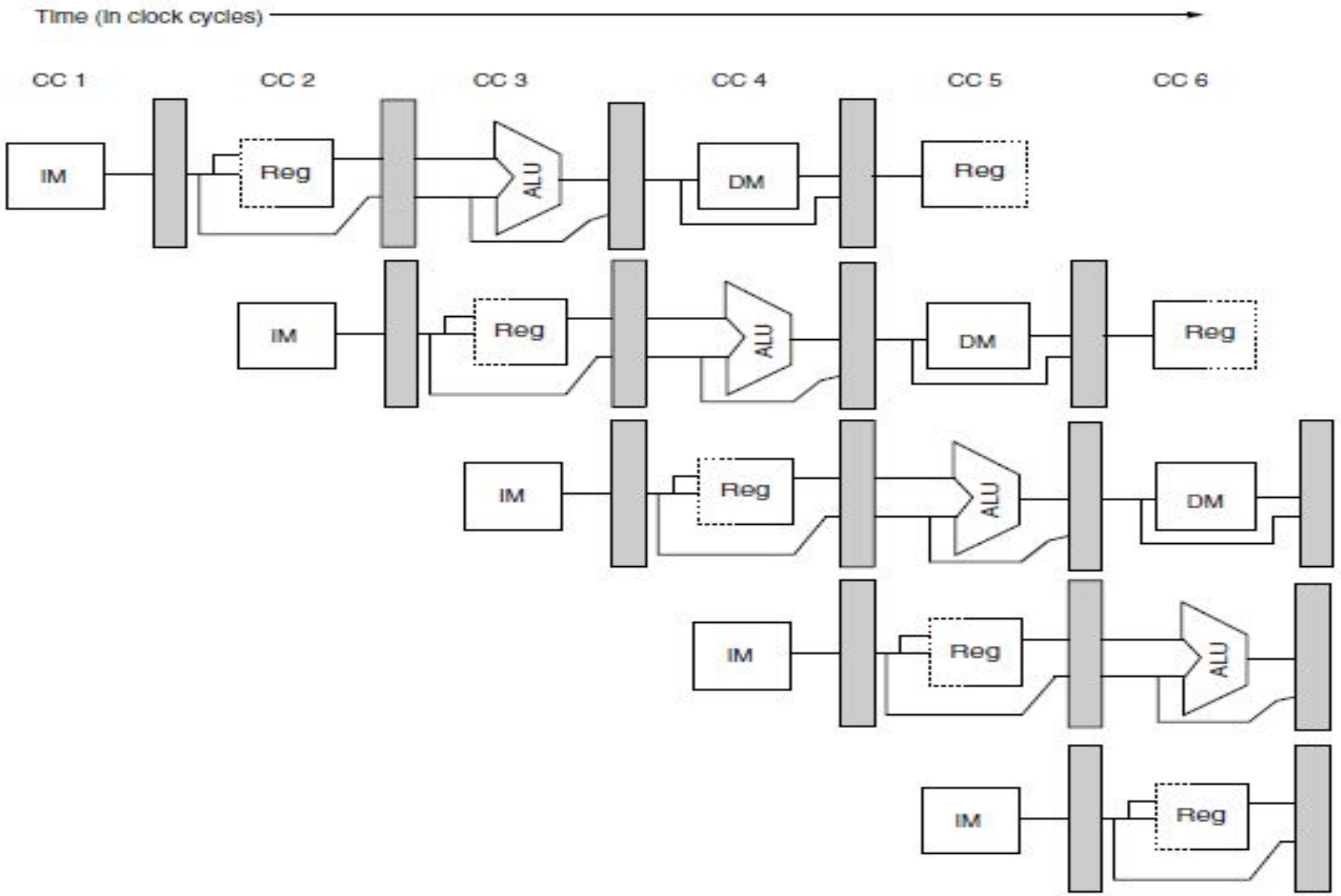
Simple RISC pipeline

	Clock Number								
Instruction number	1	2	3	4	5	6	7	8	9
Instruction i	IF	ID	EX	MEM	WB				
Instruction i+1		IF	ID	EX	MEM	WB			
Instruction i+2			IF	ID	EX	MEM	WB		
Instruction i+3				IF	ID	EX	MEM	WB	
Instruction i+4					IF	ID	EX	MEM	WB

The Pipeline Can Be Thought Of As A Series Of Data Paths Shifted In Time



The Pipeline with Pipeline Registers Between Successive Pipeline Stages



Problem:

r

Ans: The average instruction execution time on the unpipelined processor is

Average instruction execution time = Clock cycle × Average CPI

= 1 n s × ((40% + 20%) × 4 + 40% × 5)

= 1 n s × 4.4

= 4.4 ns

In the pipelined implementation, clock runs with the speed of the slowest pipe stage which will be 1 + 0.2 or 1.2 ns. Thus, the speedup from pipelining is

$$\text{Speedup from pipelining} = \frac{\text{Average instruction time unpipelined}}{\text{Average instruction time pipelined}}$$

= 4.4ns/1.2ns

= 3.7 times

The Major Hurdle of Pipelining—Pipeline Hazards`

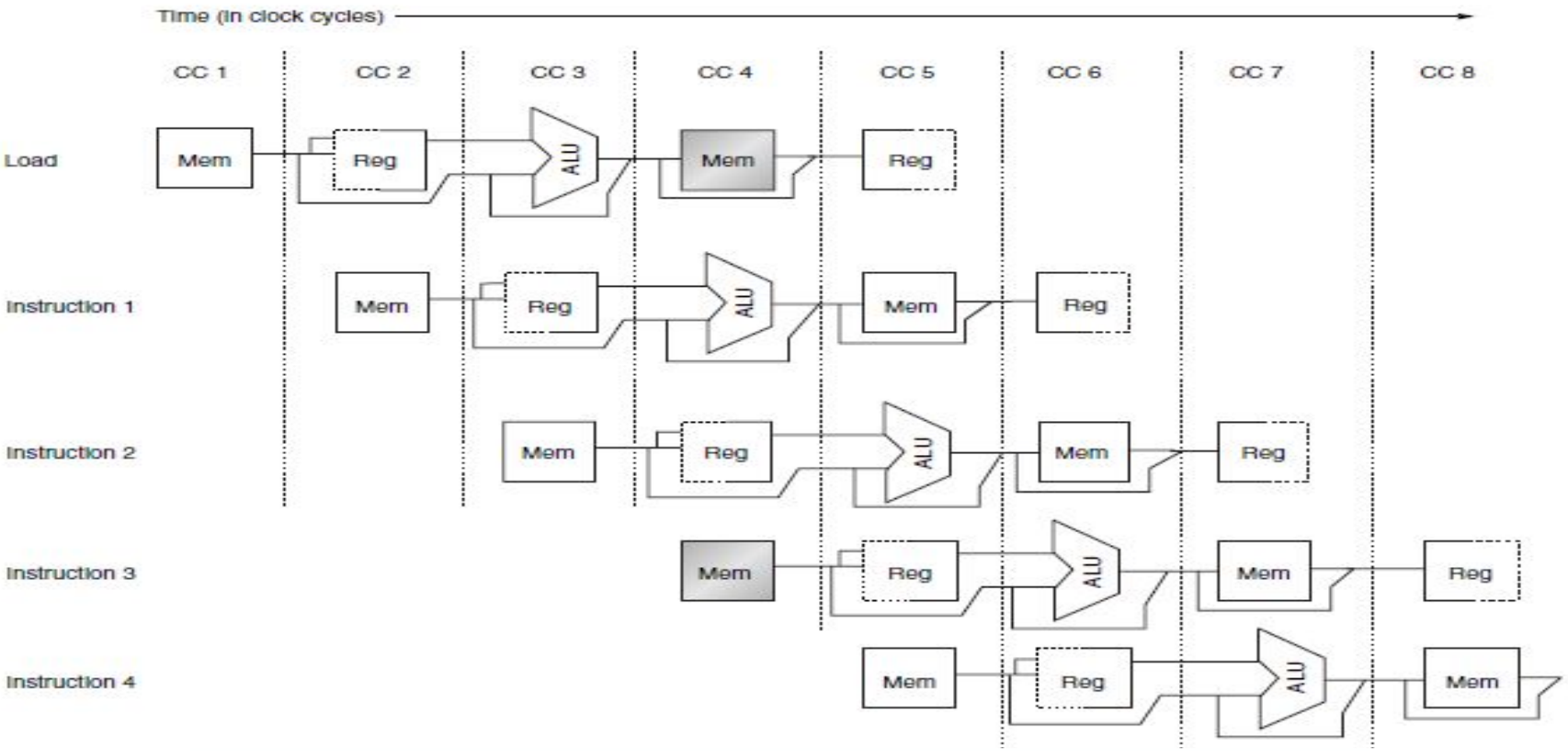
There are three classes of hazards:

1. *Structural hazards*
2. *Data hazards*
3. *Control hazards*

Structural hazards

- Conflict for use of a resource
- In MIPS pipeline with a single memory
  - Load/store requires data access
  - Instruction fetch would have to *stall for that cycle*
    - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require separate instruction/data memories
  - Or separate instruction/data caches

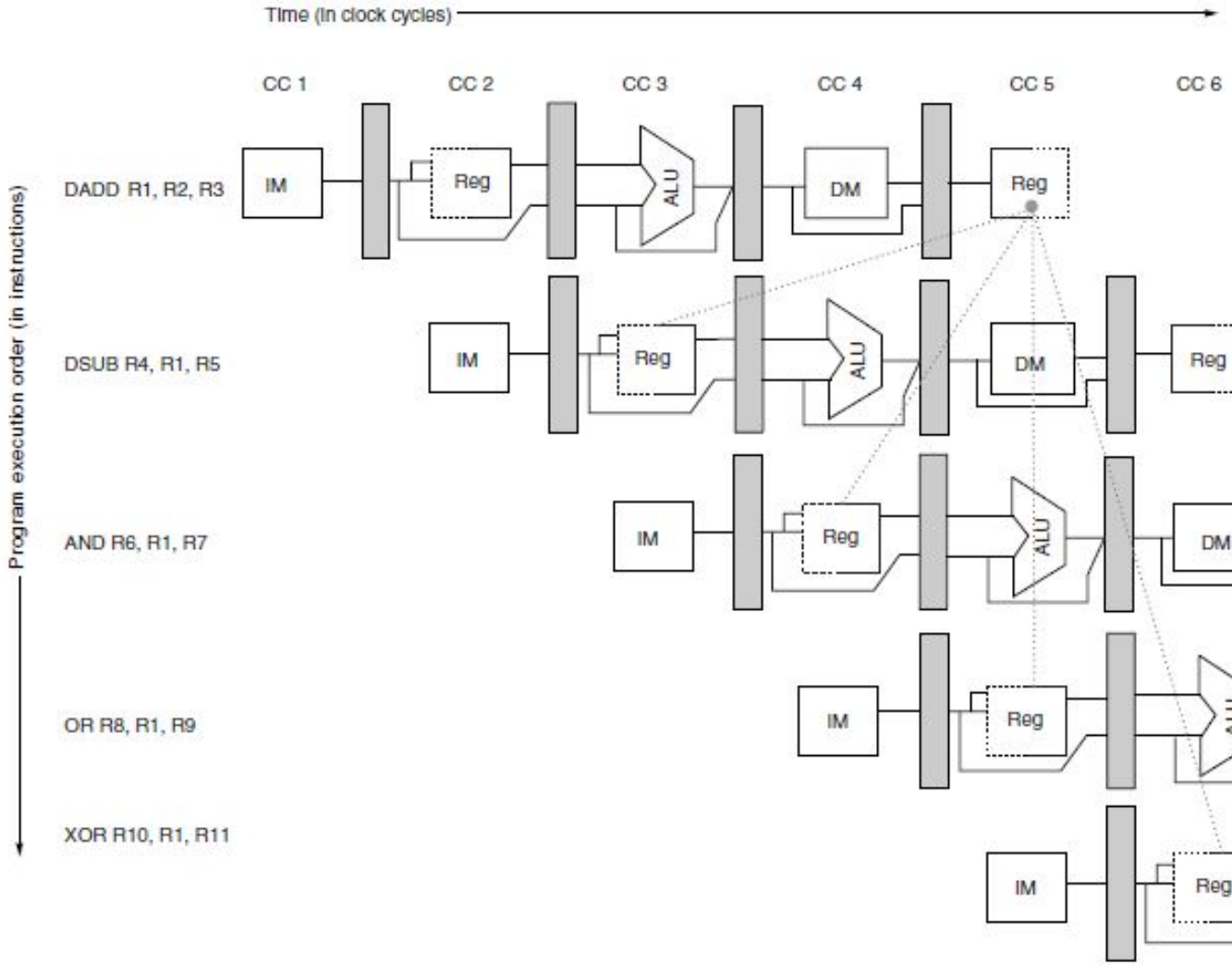
Structural hazards



	Clock cycle number									
Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Load instruction	IF	ID	EX	MEM	WB					
Instruction $i+1$		IF	ID	EX	MEM	WB				
Instruction $i+2$			IF	ID	EX	MEM	WB			
Instruction $i+3$				stall	IF	ID	EX	MEM	WB	
Instruction $i+4$						IF	ID	EX	MEM	WB
Instruction $i+5$							IF	ID	EX	MEM
Instruction $i+6$								IF	ID	EX

Data hazards

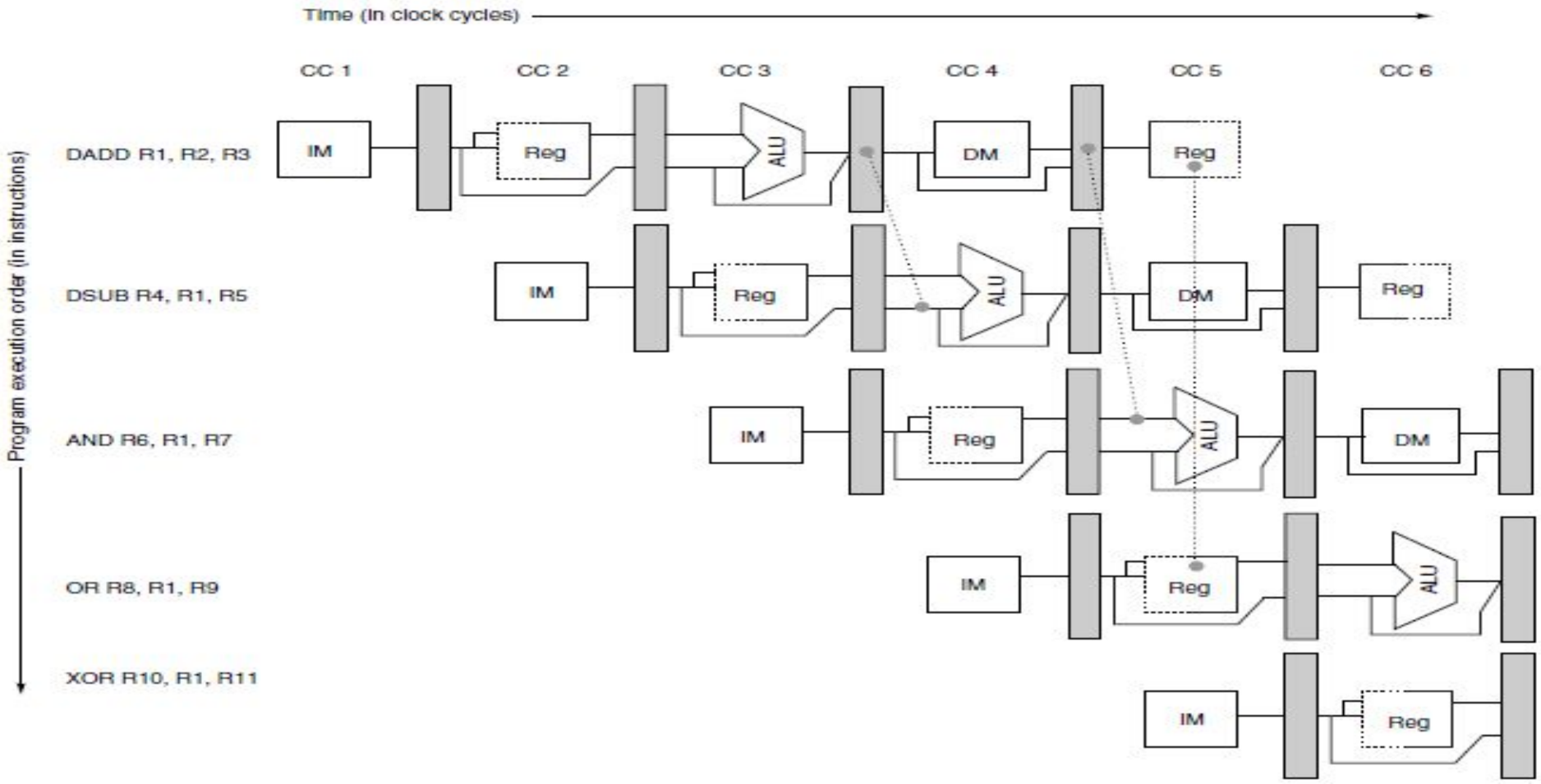
An instruction depends on completion of data access by a previous instruction



DADD **R1**,R2,R3
DSUB R4,**R1**,R5
AND R6,**R1**,R7
OR R8,**R1**,R9
XOR R10,**R1**,R11

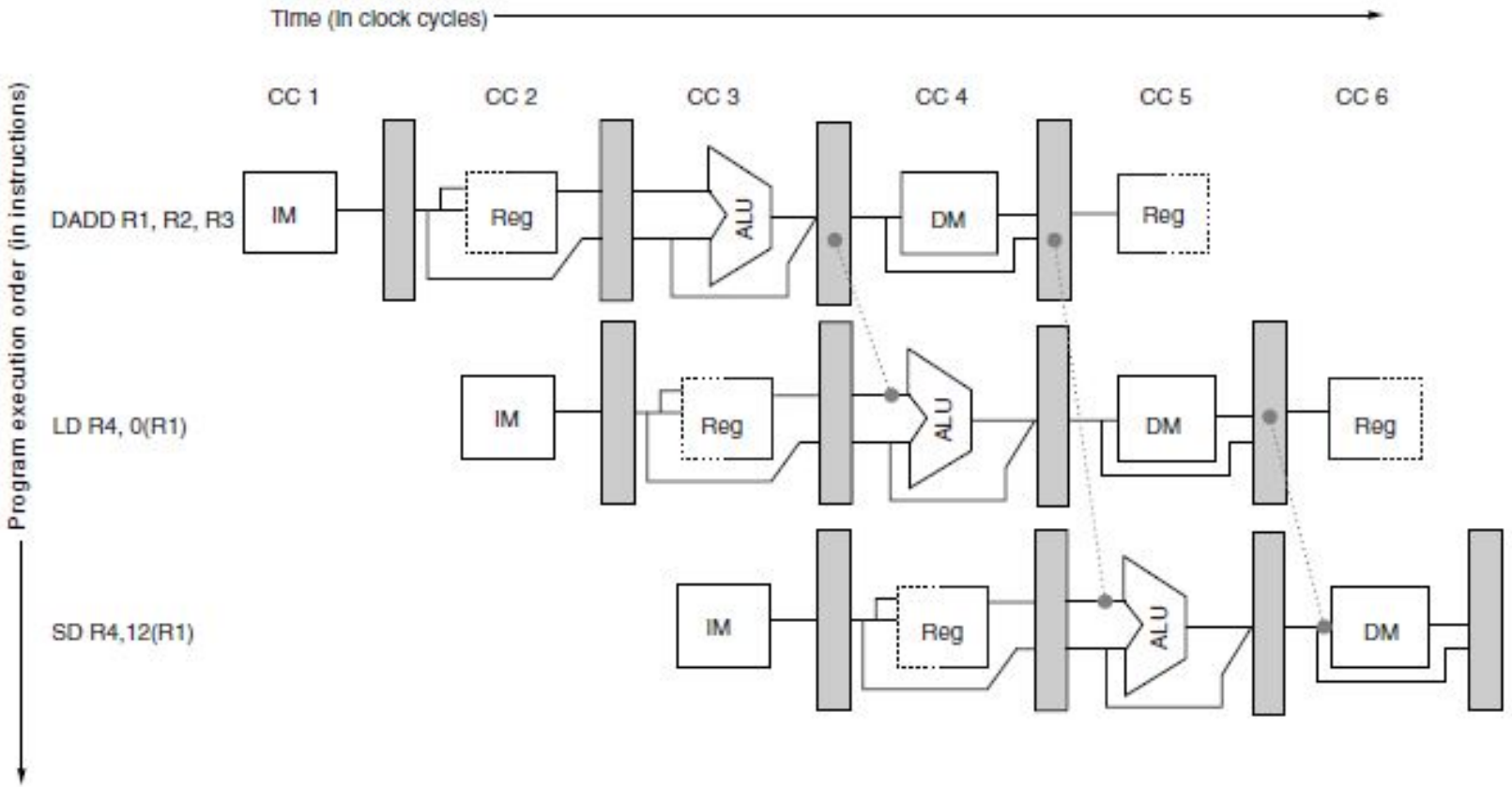
Minimizing Data Hazard Stalls by Forwarding(Bypassing)

- Use result when it is computed
  - Don’t wait for it to be stored in a register
  - Requires extra connections in the datapath



Minimizing Data Hazard Stalls by Forwarding(Bypassing)

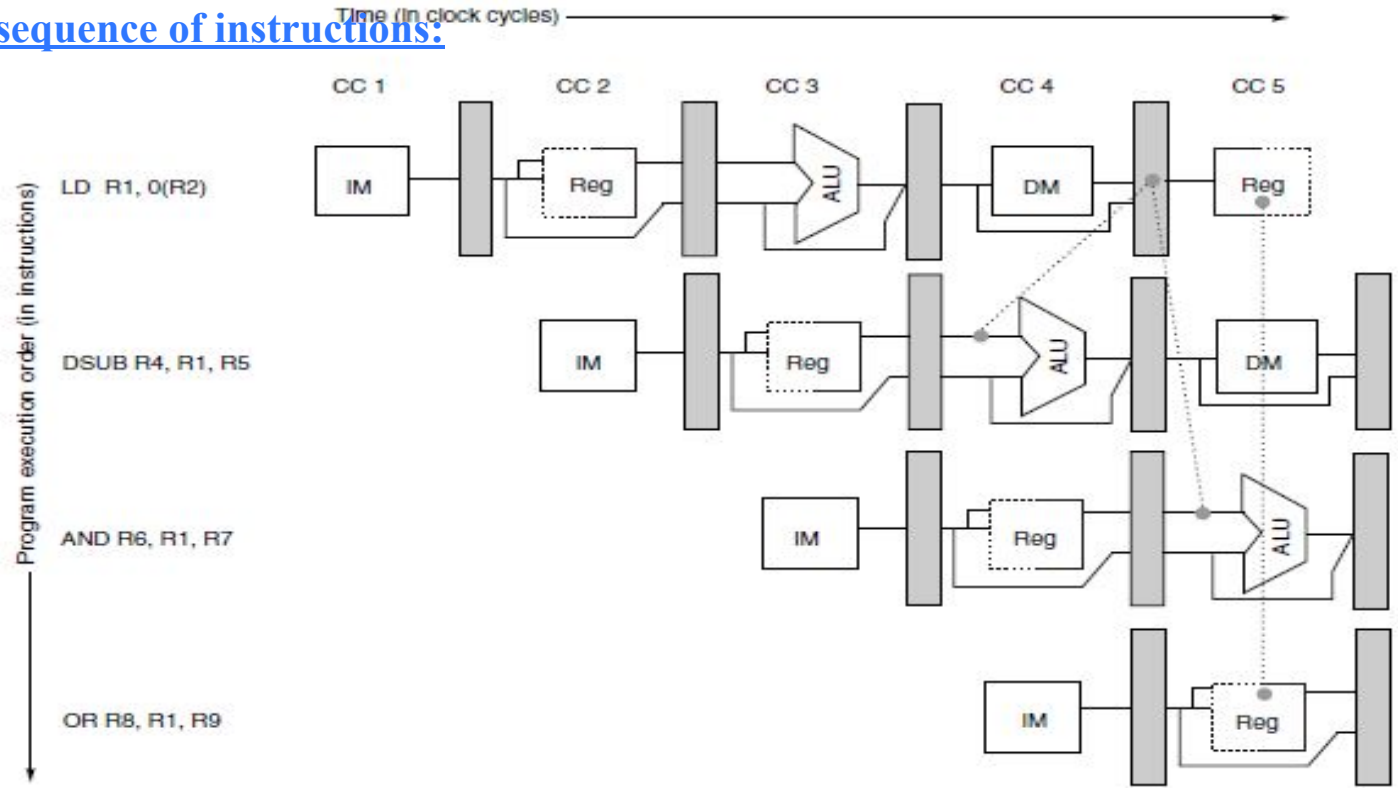
- Consider the following sequence:
  - DADD R1,R2,R3
  - LD R4,0(R1)
  - SD R4,12(R1)



Data Hazards Requiring Stalls

Consider the following sequence of instructions:

LD R1,0(R2)
DSUB R4,R1,R5
AND R6,R1,R7
OR R8,R1,R9



LD	R1,0(R2)	IF	ID	EX	MEM	WB			
DSUB	R4,R1,R5		IF	ID	EX	MEM	WB		
AND	R6,R1,R7			IF	ID	EX	MEM	WB	
OR	R8,R1,R9				IF	ID	EX	MEM	WB

LD	R1,0(R2)	IF	ID	EX	MEM	WB			
DSUB	R4,R1,R5		IF	ID	stall	EX	MEM	WB	
AND	R6,R1,R7			IF	stall	ID	EX	MEM	WB
OR	R8,R1,R9				stall	IF	ID	EX	MEM	WB

Performance of Pipelines with Stalls

- The actual speedup from pipelining

$$\text{Speedup from pipelining} = \frac{\text{Average instruction time unpipelined}}{\text{Average instruction time pipelined}}$$

$$\text{CPU time} = \text{CPU clock cycl} = \frac{\text{CPI unpipelined}}{\text{CPI pipelined}} \times \frac{\text{Clock cycle unpipelined}}{\text{Clock cycle pipelined}}$$

$$\begin{aligned}\text{CPI pipelined} &= \text{Ideal CPI} + \text{Pipeline stall clock cycles per instruction} \\ &= 1 + \text{Pipeline stall clock cycles per instruction}\end{aligned}$$

$$\text{Speedup} = \frac{\text{CPI unpipelined}}{1 + \text{Pipeline stall cycles per instruction}}$$

$$\text{Speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall cycles per instruction}}$$

$$= \frac{1}{1 + \text{Pipeline stall cycles per instruction}} \times \frac{\text{Clock cycle unpipelined}}{\text{Clock cycle pipelined}}$$

$$\text{Clock cycle pipelined} = \frac{\text{Clock cycle unpipelined}}{\text{Pipeline depth}}$$

$$\text{Pipeline depth} = \frac{\text{Clock cycle unpipelined}}{\text{Clock cycle pipelined}}$$

$$\begin{aligned}\text{Speedup from pipelining} &= \frac{1}{1 + \text{Pipeline stall cycles per instruction}} \times \frac{\text{Clock cycle unpipelined}}{\text{Clock cycle pipelined}} \\ &= \frac{1}{1 + \text{Pipeline stall cycles per instruction}} \times \text{Pipeline depth}\end{aligned}$$

Branch Hazards

- Branch determines flow of control
  - Fetching next instruction depends on branch outcome
  - Pipeline can’t always fetch correct instruction
    - Still working on ID stage of branch
- In MIPS pipeline
  - Need to compare registers and compute target early in the pipeline
  - Add hardware to do it in ID stage

Reducing Pipeline Branch Penalties

1-cycle stall in the five-stage pipeline(freeze or flush)

Branch instruction	IF	ID	EX	MEM	WB		
Branch successor		IF	IF	ID	EX	MEM	WB
Branch successor + 1				IF	ID	EX	MEM
Branch successor + 2					IF	ID	EX

predicted-not-taken or predicted untaken scheme

Untaken branch instruction	IF	ID	EX	MEM	WB			
Instruction $i + 1$		IF	ID	EX	MEM	WB		
Instruction $i + 2$			IF	ID	EX	MEM	WB	
Instruction $i + 3$				IF	ID	EX	MEM	WB
Instruction $i + 4$					IF	ID	EX	MEM	WB

Taken branch instruction	IF	ID	EX	MEM	WB			
Instruction $i + 1$		IF	idle	idle	idle	idle		
Branch target			IF	ID	EX	MEM	WB	
Branch target + 1				IF	ID	EX	MEM	WB
Branch target + 2					IF	ID	EX	MEM	WB

Reducing Pipeline Branch Penalties

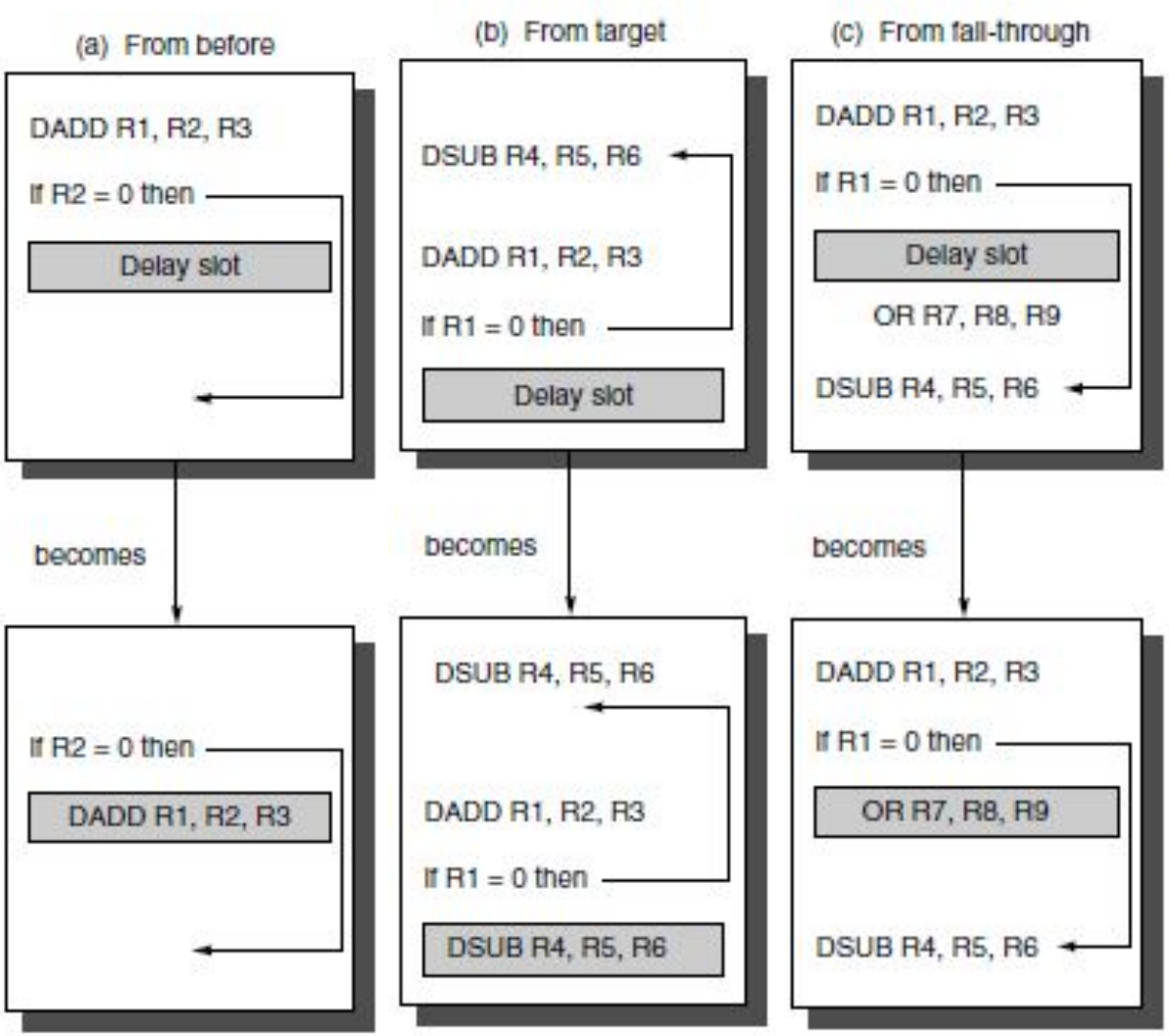
- **Scheduling the branch delay slot**
  - *branch instruction*
  - *sequential successor*
  - *branch target if taken*

Untaken branch instruction	IF	ID	EX	MEM	WB				
Branch delay instruction ($i + 1$)		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB

Taken branch instruction	IF	ID	EX	MEM	WB				
Branch delay instruction ($i + 1$)		IF	ID	EX	MEM	WB			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

Reducing Pipeline Branch Penalties

- **Scheduling the branch delay slot**



Performance of Branch Schemes

$$\text{Pipeline speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall cycles from branches}}$$

Pipeline stall cycles from branches = Branch frequency × Branch penalty

$$\text{Pipeline speedup} = \frac{\text{Pipeline depth}}{1 + \text{Branch frequency} \times \text{Branch penalty}}$$

Reducing cost of Branches through Prediction

- **Static branch prediction**
- **Dynamic branch prediction**